

LAB 4

SysTick Timer

Objectives:

WEEK 10

- To program the timer in TI Tiva development board.

Material:

- Keil IDE
- TI Tiva development board
- A Terminal software like Tera Term or Putty.
- Wires
- A computer
- Bread board
- 1602 LCD (optional)

Pre-Lab Activity

In **system_TM4C123.c** file, change **#define CLOCK_SETUP 1** to **#define CLOCK_SETUP 0**.

Lab Activity 1- Implementing a 500ms Delay and LED Blinking Using SysTick on [10 points]

Develop a program that utilizes the **SysTick timer** to create a precise **500-millisecond delay** and use it to toggle the **blue LED at a frequency of 2 Hz** (i.e., blinking 2 times per second).

Lab Activity 2- Blink LED every 2seconds [20 points]

Change the program in Activity 1 to blink LED once every 2seconds. Explain the limitations of SysTick timer and your approach to achieve this objective.

Lab Activity 3- Measuring Time Between Two Button Presses Using SysTick Timer [20 points]

Objective:

Develop a program that measures the time interval between two button presses using the SysTick timer in the TIVA C (TM4C123G) microcontroller.

Instructions:**1. Configure SysTick Timer:**

- Initialize the SysTick timer with a clock source of **16 MHz**.
- Set it to count down continuously.

2. Configure GPIO for Button Input:

- Use **Port F** for button inputs (e.g., **PF4** and **PF0**).
- Enable internal pull-up resistors since the buttons are active-low.

3. Detect Button Press Events:

- Wait for the first button press and store the current SysTick value.
- Wait for the second button press and store the new SysTick value.

4. Calculate the Time Difference:

- Compute the elapsed time in microseconds using the SysTick clock frequency.

5. Display the Result:

- Send the measured time difference to a **UART terminal** or display it on an **LCD**.

Lab Activity 4. Challenge Question [20 points]

A student wants to create a program that performs two periodic tasks:

- Task A runs every 1 ms (sensor sampling)
- Task B runs every 100 ms (status update)

The student writes the following approach:

```
while(1){
    delay_ms(1);        // using SysTick
    TaskA();

    if(counter == 100){
        TaskB();
        counter = 0;
    }
    counter++;
}
```

1. Will Task A execute exactly every **1 ms**? Explain why or why not.
2. If TaskA() takes **0.2 ms to execute**, what is the actual period of Task A?
3. After 1 second of execution, will Task B run exactly 10 times?

Explain any timing error.

4. What is the main limitation of using SysTick with **busy-wait delays** for periodic tasks?
5. Propose a better design using SysTick that ensures:
 - a. accurate periodic execution
 - b. no timing drift

(Hint: think about **interrupts or flags**)

Questions [30 points]

1. Explain how you configured the SysTick timer to generate a 500 ms delay using the 16 MHz system clock. Include the register settings and the mathematical calculation to determine the SysTick reload value.
2. How would you modify the code to achieve 10 Hz blinking rate?
3. If the SysTick timer was set to a different clock source (for example, a slower clock (10MHz)), how would that impact the blinking frequency? How would you adjust the SysTick settings to maintain a 10 Hz blinking rate?
4. After capturing the SysTick values during both button presses, explain how you calculate the elapsed time. Provide the formula used for converting the SysTick counter difference into microseconds.
5. How would your program behave if the time interval between button presses exceeds the maximum value that SysTick can measure (e.g., if the timer rolls over)? What changes would you make to handle longer time intervals?

